ACTIVITY#2

```
def printParents(node, adj, parent):
    # current node is Root, thus, has no parent
    if (parent == 0):
        print(node, "->Root")
    else:
        print(node, "->", parent)

    # Using DFS
    for cur in adj[node]:
        if (cur != parent):
            printParents(cur, adj, node)

# Function to print the children of each node
def printChildren(Root, adj):
    # Queue for the BFS
    q = []
    q.append(Root)

    # visit array to keep track of visited nodes
    vis = [0] * len(adj)

    # BFS
    while (len(q) > 0):
        node = q[0]
        q.pop(0)
        vis[node] = 1
        print(node, "-> ", end=" ")

        for cur in adj[node]:
            if (vis[cur] == 0):
                print(cur, " ", end=" ")
                q.append(cur)
        print("\n")

# Function to print the leaf nodes
def printLeafNodes(Root, adj):
    # Leaf nodes have only one edge and are not the root
    for i in range(0, len(adj)):
        if (len(adj[i]) == 1 and i != Root):
            print(i, end=" ")
    print("\n")

# Function to print the degrees of each node
def printDegrees(Root, adj):
    for i in range(1, len(adj)):
        print(i, ": ", end=" ")
        # Root has no parent, so its degree equals its edge count
        if (i == Root):
            print(len(adj[i]))
        else:
            print(len(adj[i]) - 1)

# ── Driver code ─────────────────────────────────────────
# Number of nodes
N = 7
Root = 1

# Adjacency list to store the tree
adj = []
for i in range(0, N + 1):
    adj.append([])

# Creating the tree
adj[1].append(2); adj[2].append(1)
adj[1].append(3); adj[3].append(1)
adj[1].append(4); adj[4].append(1)
adj[2].append(5); adj[5].append(2)
adj[2].append(6); adj[6].append(2)
adj[4].append(7); adj[7].append(4)

# Printing the parents of each node
print("The parents of each node are:")
```

```
printParents(Root, adj, 0)

# Printing the children of each node
print("\nThe children of each node are:")
printChildren(Root, adj)

# Printing the leaf nodes in the tree
print("The leaf nodes of the tree are:")
printLeafNodes(Root, adj)

# Printing the degrees of each node
print("The degrees of each node are:")
printDegrees(Root, adj)
```

```
The parents of each node are:
1 ->Root
2 -> 1
5 -> 2
6 -> 2
3 -> 1
4 -> 1
7 -> 4

The children of each node are:
1 ->  2   3   4

2 ->  5   6

3 ->

4 ->  7

5 ->

6 ->

7 ->

The leaf nodes of the tree are:
3 5 6 7

The degrees of each node are:
1 :  3
2 :  2
3 :  0
4 :  1
5 :  0
6 :  0
7 :  0
```

ACTIVITY#1

```
# Graph definition (adjacency list)
graph = {
    '5': ['3', '7'],
    '3': ['2', '4'],
    '7': ['8'],
    '2': [],
    '4': ['8'],
    '8': []
}

visited = []  # List for visited nodes
queue = []    # Initialize a queue

def bfs(visited, graph, node):  # Function for BFS
    visited.append(node)
    queue.append(node)

    while queue:  # Creating loop to visit each node
        m = queue.pop(0)
        print(m, end=" ")

        for neighbour in graph[m]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

# Driver Code
```

```
print("Following is the Breadth-First Search")
bfs(visited, graph, '5')  # Function calling
```

```
Following is the Breadth-First Search
5 3 7 2 4 8
```

```
# Graph based on the provided image
graph = {
    'A': ['B', 'E', 'C'],
    'B': ['A', 'E', 'D'],
    'E': ['A', 'B'],
    'C': ['A', 'F', 'G'],
    'D': ['B'],
    'F': ['C'],
    'G': ['C']
}

def bfs(graph, start, goal):
    visited = []
    queue = []

    visited.append(start)
    queue.append([start])  # Store path instead of just node

    if start == goal:
        print(f"Start and Goal are the same node: {start}")
        return

    while queue:
        path = queue.pop(0)
        node = path[-1]  # Get last node in path

        for neighbour in graph[node]:
            if neighbour not in visited:
                new_path = list(path)
                new_path.append(neighbour)
                queue.append(new_path)
                visited.append(neighbour)

                # Goal found
                if neighbour == goal:
                    print(f"  Path Found: {' -> '.join(new_path)}")
                    print(f"  Nodes Visited: {visited}")
                    return

    print("  No path found between", start, "and", goal)


# ── Testing different Start & Goal combinations ──────────
test_cases = [
    ('A', 'G'),   # Root to leaf
    ('D', 'F'),   # Leaf to leaf
    ('B', 'C'),   # Mid to mid
    ('G', 'D'),   # Reverse traversal
    ('E', 'F'),   # Cross branch
]

for start, goal in test_cases:
    print(f"\nStart: {start}  |  Goal: {goal}")
    bfs(graph, start, goal)
```

```
Start: A  |  Goal: G
  Path Found: A -> C -> G
  Nodes Visited: ['A', 'B', 'E', 'C', 'D', 'F', 'G']

Start: D  |  Goal: F
  Path Found: D -> B -> A -> C -> F
  Nodes Visited: ['D', 'B', 'A', 'E', 'C', 'F']

Start: B  |  Goal: C
  Path Found: B -> A -> C
  Nodes Visited: ['B', 'A', 'E', 'D', 'C']

Start: G  |  Goal: D
  Path Found: G -> C -> A -> B -> D
  Nodes Visited: ['G', 'C', 'A', 'F', 'B', 'E', 'D']
```

```
Start: E  |  Goal: F
  Path Found: E -> A -> C -> F
  Nodes Visited: ['E', 'A', 'B', 'C', 'D', 'F']
```

```
# Graph based on the provided image
graph = {
    'A': ['B', 'E', 'C'],
    'B': ['A', 'E', 'D'],
    'E': ['A', 'B'],
    'C': ['A', 'F', 'G'],
    'D': ['B'],
    'F': ['C'],
    'G': ['C']
}

visited = []  # List for visited nodes
queue = []    # Initialize a queue

def bfs(visited, graph, node):  # Function for BFS
    visited.append(node)
    queue.append(node)

    while queue:  # Creating loop to visit each node
        m = queue.pop(0)
        print(m, end=" ")

        for neighbour in graph[m]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

# Driver Code
print("Following is the Breadth-First Search")
bfs(visited, graph, 'A')  # Starting from root node A
```

```
Following is the Breadth-First Search
A B E C D F G
```